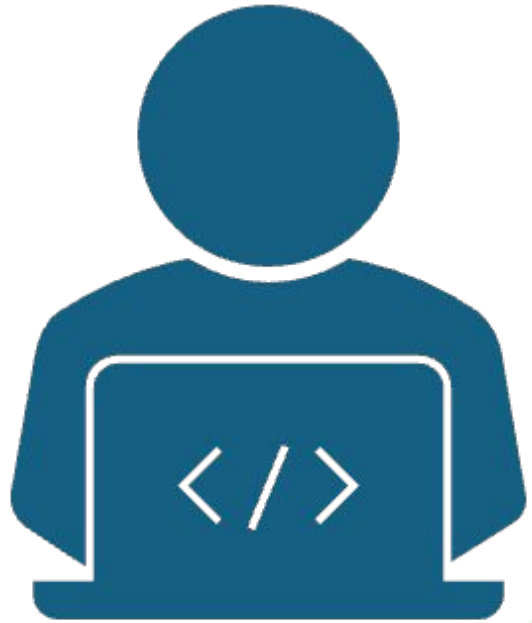
Intro to Technical Interviews

Attendance

linktr.ee/CODE.CRUNCH

Technical Interview

- A technical interview evaluates problem-solving, coding, and algorithmic skills for software engineering roles.
- Interviewers assess your logical thinking, code efficiency, and ability to communicate your approach.
- You’re expected to write clean, functional code while explaining your thought process clearly.



Technical Interview Process

Problem Statement:

You are presented with a problem to solve within a limited time (often 30–45 minutes).

Clarifying Questions:

Ask any clarifying questions to ensure you fully understand the problem before you start coding.

Plan Your Approach:

Think through potential solutions and choose the most efficient one.

Write Code:

Begin coding the solution while explaining your thought process to the interviewer.

Testing & Debugging:

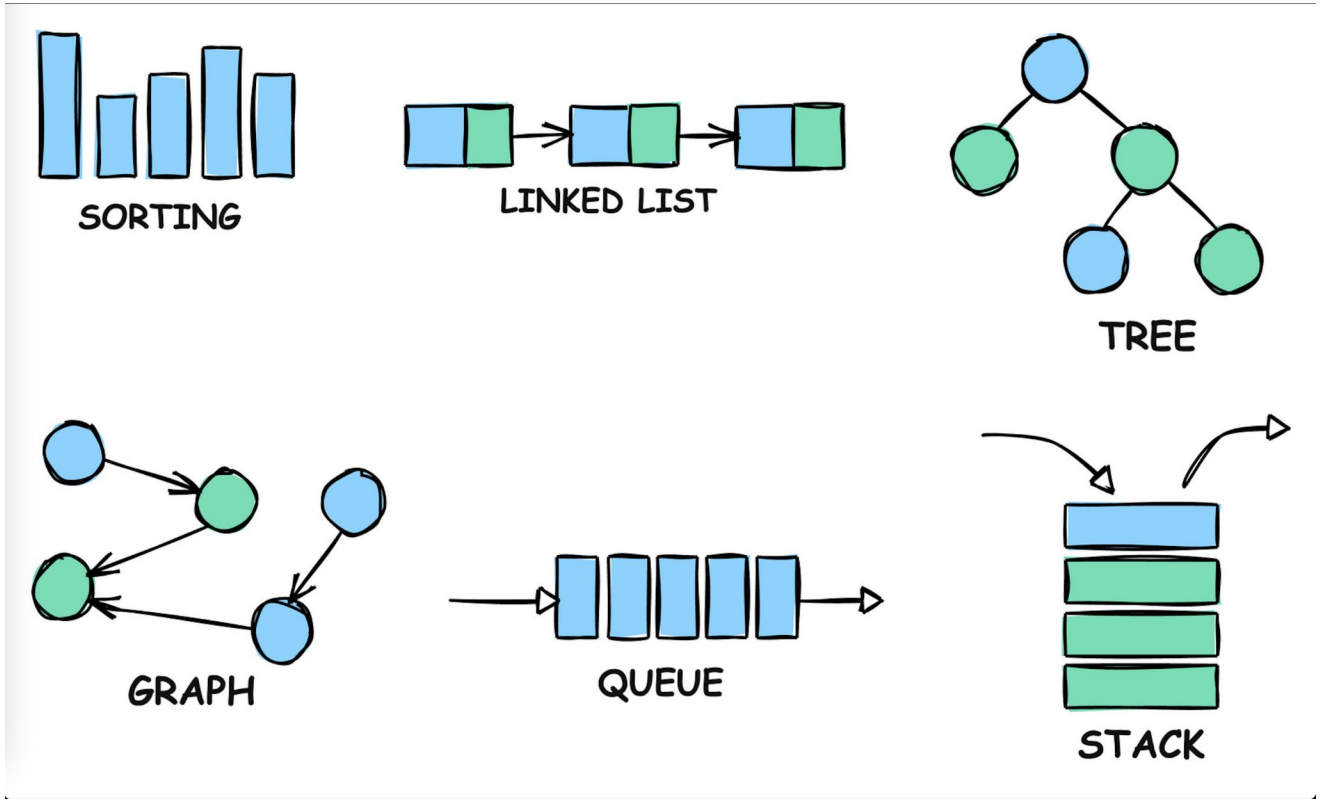
Test your solution with edge cases and inputs to ensure correctness.

Time and Space Complexity Analysis:

Analyze the time and space complexity of your solution using Big O notation.

Preparing for Technical Interviews

- Data Structures and Algorithm problems form the foundation of most technical interviews.
- Many interview questions are based on common DSA patterns and familiarity with them speeds up problem-solving.
- Understanding DSA and Time and Space complexity allows you to solve problems faster and more efficiently.



LeetCode & Hackerrank

LeetCode and **HackerRank** are online platforms that offer coding challenges and practice problems designed to prepare candidates for technical interviews.

- **Why They're Important:** They offer DSA problems similar to those asked in real interviews.
- Both websites offer hundreds of different questions of different difficulty levels, with guided plans and explanations that help you learn and improve.

Time and Space Complexity

Time and space complexity are key for understanding how algorithms perform as input size grows. **Time complexity** refers to the amount of time an algorithm takes, while **space complexity** focuses on the amount of memory it requires.

Big O Notation

Big O describes how the performance of an algorithm scales as the input size increases. It helps explain the time it takes to run an algorithm and the space (memory) it uses.

Why is it Important?

- **Compare algorithms:** Easily see which algorithm is faster or uses less memory.
- **Optimize code:** Helps improve the efficiency of your programs.
- **Technical interviews:** Understanding Big O is key for solving coding challenges.

Big O Notation

Most Common Big O Notations:

- O(1) : Constant time - The algorithm takes the **same amount of time** regardless of input size.
- O(n) : Linear time - The running time grows **directly** with the size of the input.
- O(n^2) : Quadratic time - The running time grows **quadratically** with the input size.

```
# Example
def sum_elements(arr):
    total = 0
    for num in arr:
        total += num
    return total
```

Constant Time

- Variable Assignment
- Arithmetic Operations
- Comparisons
- Accessing Elements in a List by Index
- Boolean Operations
- Return Statements

```python
def constant_time_operations():

    # Variable assignment: O(1)
    x = 10

    # Arithmetic operation: O(1)
    y = x + 5

    # Comparison: O(1)
    is_equal = (y == 15)

    # Accessing list element by index: O(1)
    my_list = [1, 2, 3, 4]
    value = my_list[2]

    # Boolean operation: O(1)
    result = is_equal and value == 3

    # Return statement: O(1)
    return result
```

Linear Time

Time Complexity: The function iterates through the list once, making it **O(n)**, where *n* is the number of elements in the list. This is **linear time complexity**, meaning the execution time increases proportionally with input size.

Space Complexity: The algorithm uses a constant amount of memory (for the total variable), so space complexity is **O(1)**.

```
def linear_time_operation(my_list):
    # Variable assignment: O(1)
    total = 0
    # Looping through a list: O(n)
    for item in my_list:
        # Arithmetic operation: O(1)
        total += item

    return total
```

Quadratic Time

```python
def print_all_pairs(my_list):
    # Outer loop iterating through each element: O(n)
    for i in range(len(my_list)):
        # Inner loop iterating through each element: O(n)
        for j in range(len(my_list)):
            # Print each pair: O(1)
            print(my_list[i], my_list[j])
```

Quadratic Time Complexity $O(n^2)$ occurs when there are two nested loops, each iterating n times, where n is the size of the input list.

In this function, for each element in the outer loop, the inner loop also runs n times, resulting in **$O(n) * O(n) = O(n^2)$** time complexity.

UMPIRE Method

U - Understand	Clarify the problem requirements. Ask questions if necessary.
M - Match	Identify a potential solution approach by matching the problem to a known algorithm or technique.
P - Plan	Plan your solution in pseudocode or steps before you write the code.
I - Implement	Write the code according to your plan.
R - Review	Review the code for mistakes or inefficiencies.
E - Evaluate	Test your solution with different cases, including edge cases. Explain time and space complexity.

Problem 1

Write a function that takes a list of integers my_list and returns the sum of all the even numbers in the list.

Example

Input: [10, 21, 32, 43, 54]

Output: 96

Explanation:

Even numbers in the list are 10, 32, and 54.

Sum of even numbers is 10 + 32 + 54 = 96

Problem 2

Write a function that takes a list of integers `lst` and returns the count of elements that are greater than the average of the list.

Example:

Input: [2, 8, 10, 4]

Output: 2

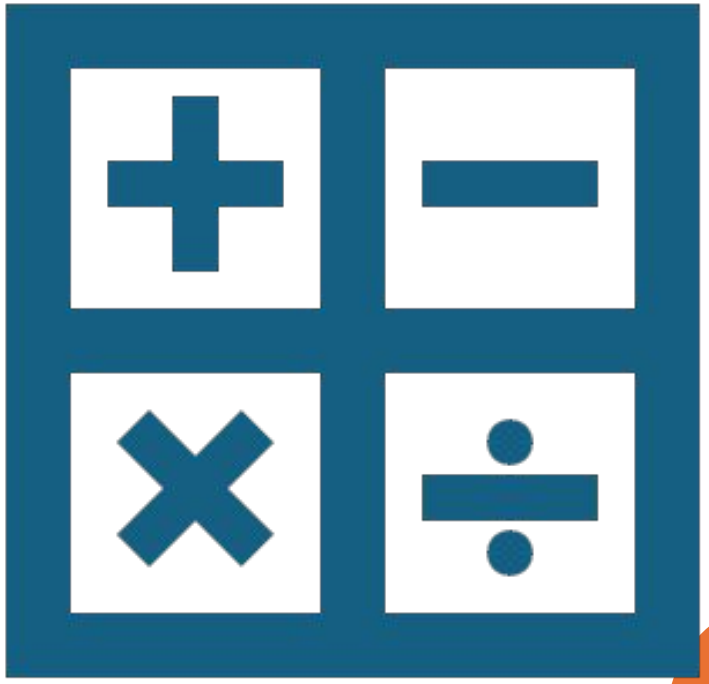
Explanation:

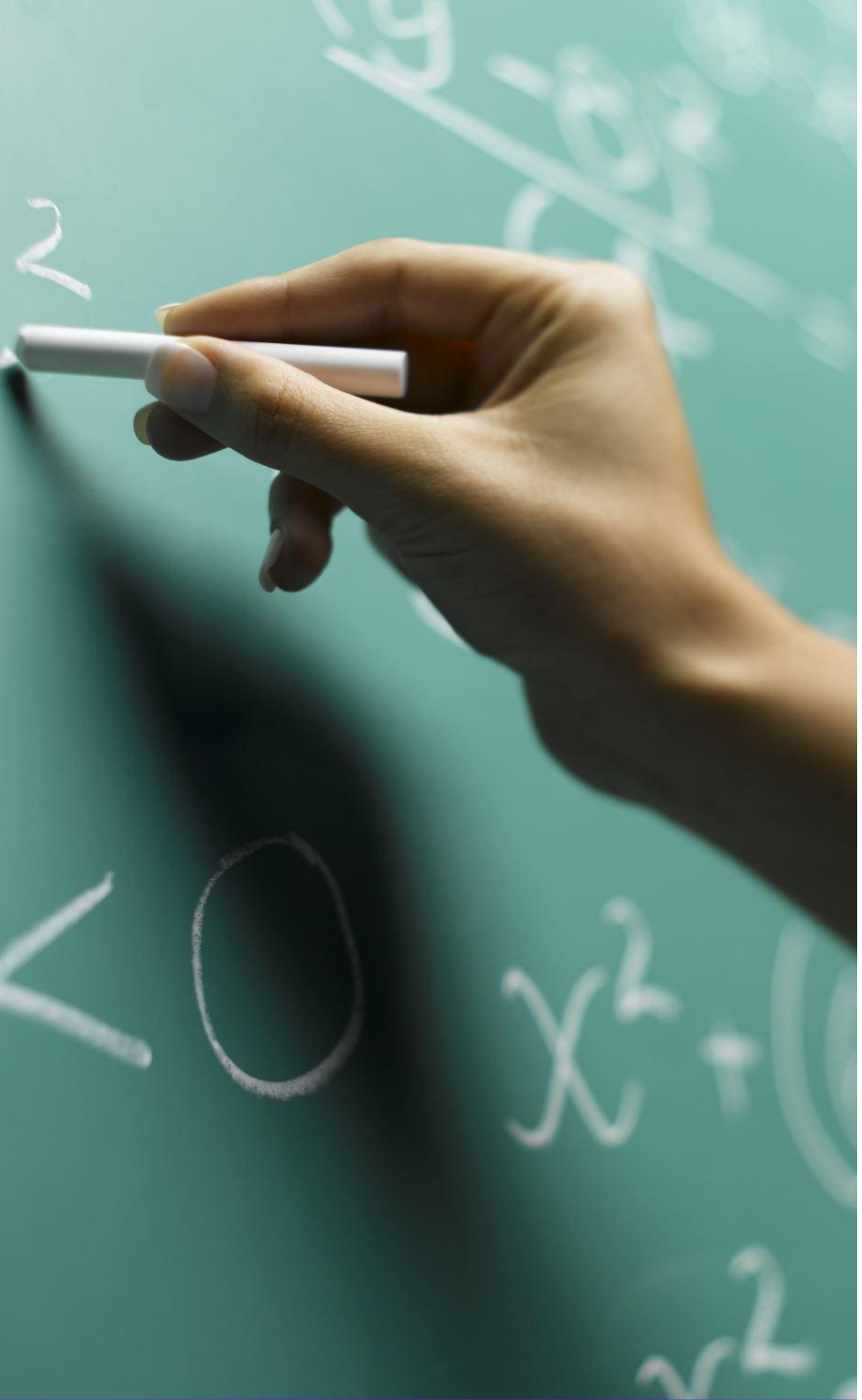
Average = (2 + 8 + 10 + 4) / 5

Average = 24 / 5 =

Average = 4.8

Elements greater than 4.8 are 8 and 10





Problem 3

Write a function that takes in a list of integers my_list and returns the difference between the smallest and largest value in the list.

Example

Input: [5,22,8,10,2]

Output: 20

Explanation:

Largest value is 22 and smallest value is 2.

22 - 2 = 20

Problem 4

Write a function that takes in an integer n as a parameter that returns a list of all divisors of n

Example:

Input: 6

Output: [1, 2, 3, 6]

Explanation:

The numbers 1, 2, 3, and 6 evenly divide the number 6, without any remainder.

Attendance

linktr.ee/CODE.CRUNCH

Join us for the next sessions

Crunch Convos: WED 2PM - 3PM
Crunch Code: FRI 1PM - 2PM

RSVP lu.ma/CODECRUNCH

Your feedback helps us improve. Share your thoughts!
Go to our website: go.fiu.edu/codecrunch